

CLI, Contenedores y Procesos

Repaso de conceptos: terminales, Docker, WSL, virtualización y fork()

A terminal window with a dark blue background and light blue text. The title bar shows three colored circles (red, yellow, green) and the text 'bash'. The terminal content shows a series of commands and their outputs: '\$ whoami' returns 'estudiante_iso', '\$ uname -a' returns 'Linux entorno-c 6.6.0', and '\$./ejecutable_procesos' returns two lines of text: 'Soy el PADRE. PID 74, hijo 75' and 'Soy el HIJO. PID 75, papá 74'. The prompt '\$ _' is visible at the bottom.

```
bash

$ whoami
estudiante_iso
$ uname -a
Linux entorno-c 6.6.0
$ ./ejecutable_procesos
Soy el PADRE. PID 74, hijo 75
Soy el HIJO. PID 75, papá 74
$ _
```

¿Qué vamos a repasar?

1

Interfaces: CLI vs GUI

2

Terminales en Windows: CMD, PowerShell, WSL

3

Procesos y la llamada fork()

4

Distros de Linux: Debian y Ubuntu

5

Docker: imágenes y contenedores

6

Virtualización: dual boot vs VirtualBox vs WSL2

7

Errores comunes y lecciones aprendidas

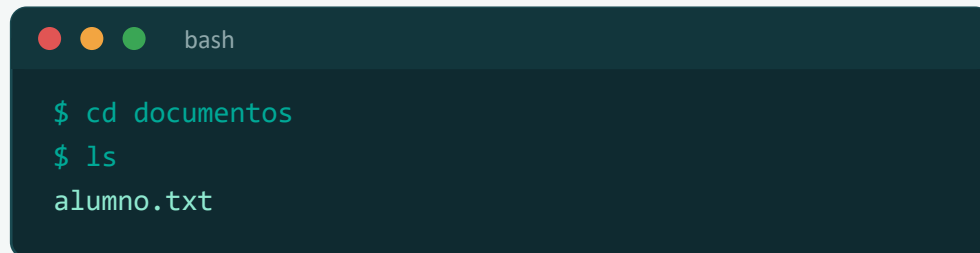
CLI vs GUI

Dos paradigmas distintos para interactuar con la computadora

CLI

Command Line Interface

- Se basa en texto: escribes comandos
- Muy rápida y scriptable (automatizable)
- Requiere memorizar sintaxis exacta
- Ejemplos: CMD, PowerShell, bash/zsh

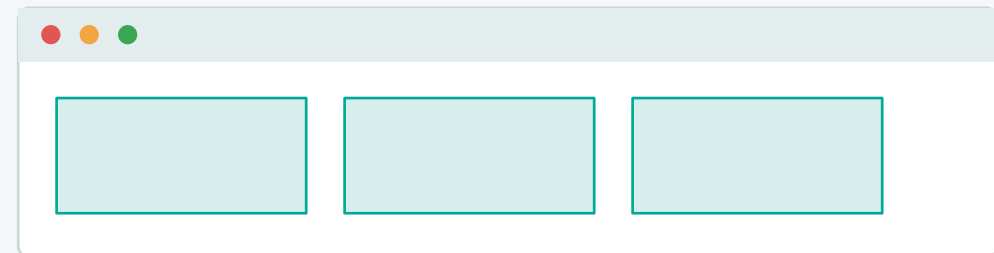
A terminal window with a dark background and light green text. The title bar shows three colored circles (red, yellow, green) and the text 'bash'. The content shows a prompt '\$' followed by the command 'cd documentos', another prompt '\$' followed by 'ls', and the output 'alumno.txt' on the next line.

```
bash
$ cd documentos
$ ls
alumno.txt
```

GUI

Graphical User Interface

- Se basa en elementos visuales
- Ventanas, íconos, menús y botones
- Intuitiva, no exige recordar comandos
- Ejemplos: Explorador de archivos, Docker Desktop



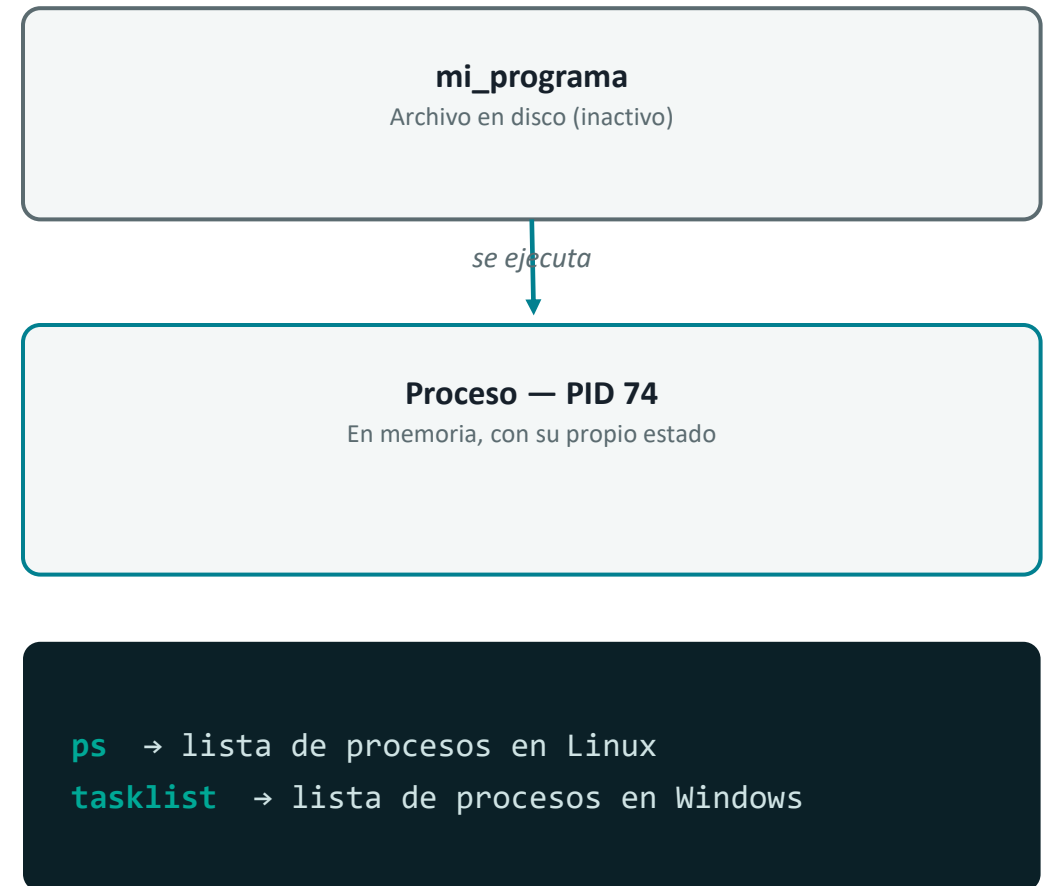
CMD vs PowerShell vs WSL

	CMD	PowerShell	WSL
Prompt	C:\Users\...>	PS C:\Users\...>	usuario@host:~\$
¿Entiende ls?	No	Sí	Sí
¿Entiende \${PWD}?	No	Sí	Sí (sintaxis Linux)
Colores de sintaxis	No	Sí	Sí
Qué es en realidad	Heredado de MS-DOS	Terminal moderna de Windows	Linux real dentro de Windows

Regla clave: cada terminal solo entiende su propia sintaxis de variables. En CMD la carpeta actual es %cd%; en PowerShell es \${PWD}. Nunca se mezclan.

¿Qué es un proceso?

- Un proceso es un programa en ejecución.
- Un programa guardado en disco (gcc, mi_programa) es solo un archivo — no hace nada por sí solo.
- En cuanto lo ejecutas, el sistema operativo le asigna un PID (Process ID) único.
- Cada proceso tiene su propia memoria y su propio estado, independiente de los demás.
- El kernel puede tener cientos de procesos corriendo a la vez (ps, tasklist).



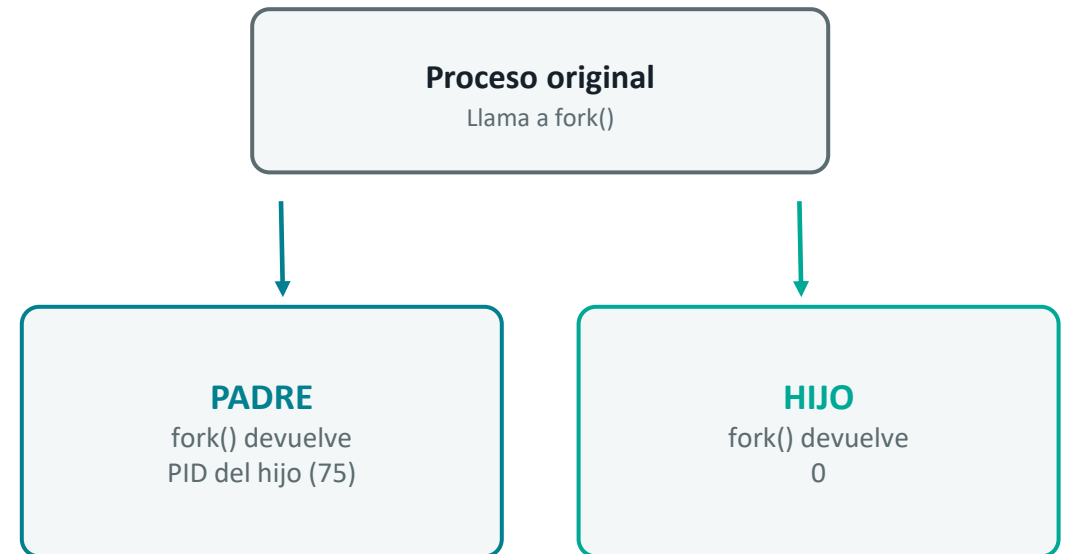
fork(): duplicar un proceso

procesos.c

```
pid_t fh = fork();

if (fh < 0) {
    // Error: no se pudo crear el hijo
} else if (fh == 0) {
    // HIJO: fork() devuelve 0
} else {
    // PADRE: fork() devuelve el PID del hijo
}
```

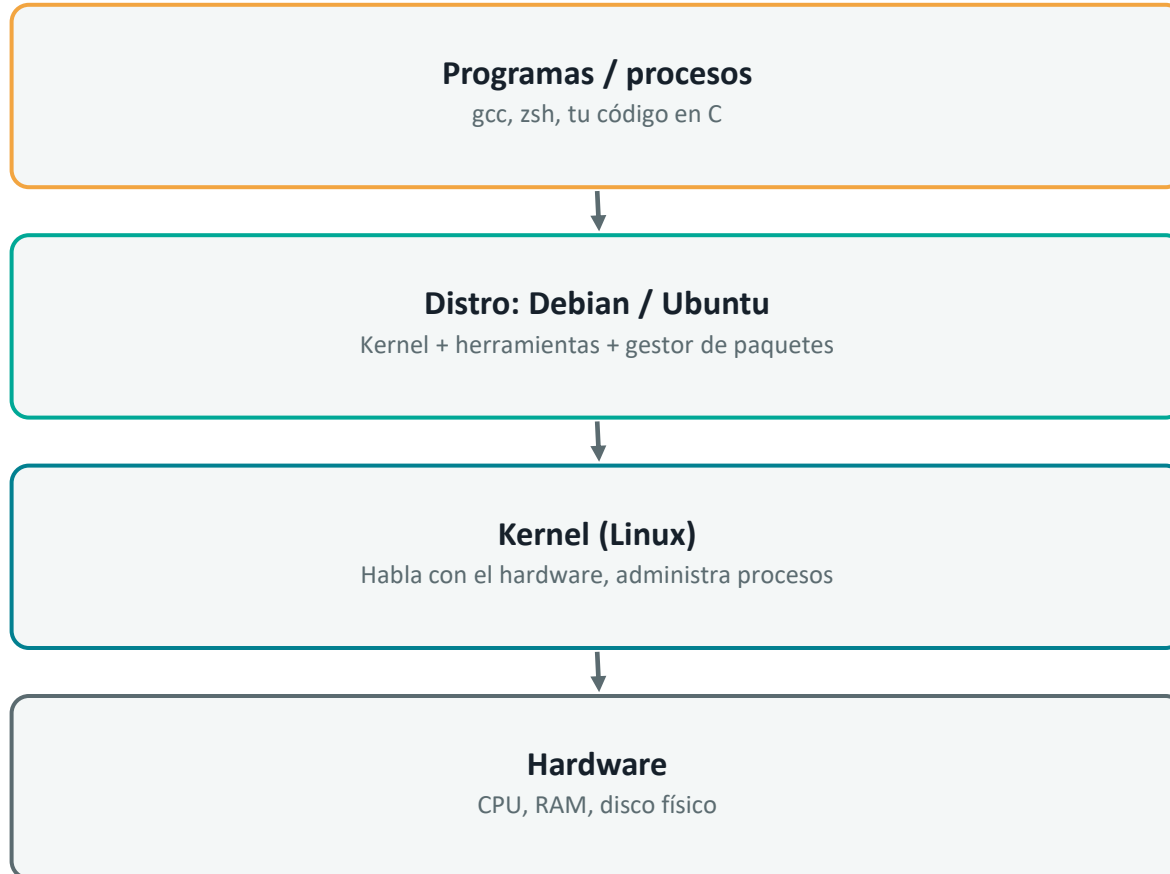
- fork() es una llamada al sistema exclusiva de Unix/Linux — por eso no existe en Windows nativo.
- Duplica el proceso que la llama: después de esa línea hay dos procesos ejecutando el mismo código.
- El valor que devuelve es la única forma de distinguir quién es quién.



bash

```
$ ./ejecutable_procesos
Soy el PADRE. Mi PID es 74
y mi hijo es 75
Soy el HIJO. Mi PID es 75
y mi papá es 74
```

Hardware, kernel y distro



¿Qué es una distro?

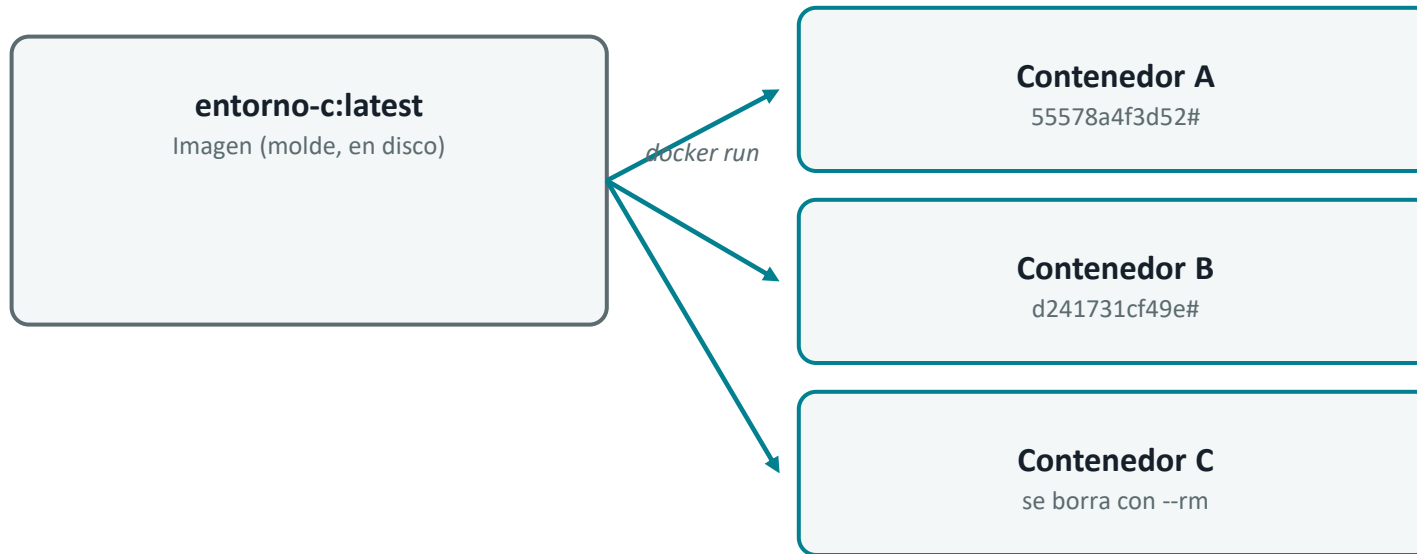
- El kernel solo no sirve de mucho para un usuario: no tiene editor, ni instalador, ni terminal amigable.
- Una distro = Linux + un conjunto de programas y configuraciones listas para usar.
- Debian y Ubuntu son distros distintas (Ubuntu está basada en Debian).

Debian "Bookworm"

= nombre en clave de Debian 12
(personajes de Toy Story)

Imagen vs contenedor

La imagen es el molde; el contenedor es la instancia en ejecución



Una sola imagen → varios contenedores desechables

Imagen

Plantilla congelada: SO base + programas + configuración, empaquetados en capas. No está corriendo.

Contenedor

Instancia en ejecución de una imagen. Puedes crear varios y tirarlos sin afectar a la imagen original.

Analogía: la imagen es el molde de galletas; el contenedor es cada galleta horneada.

docker run — flags clave

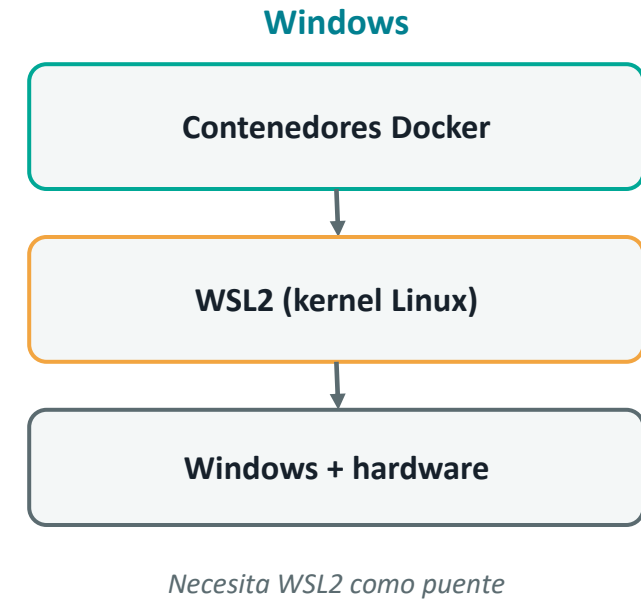
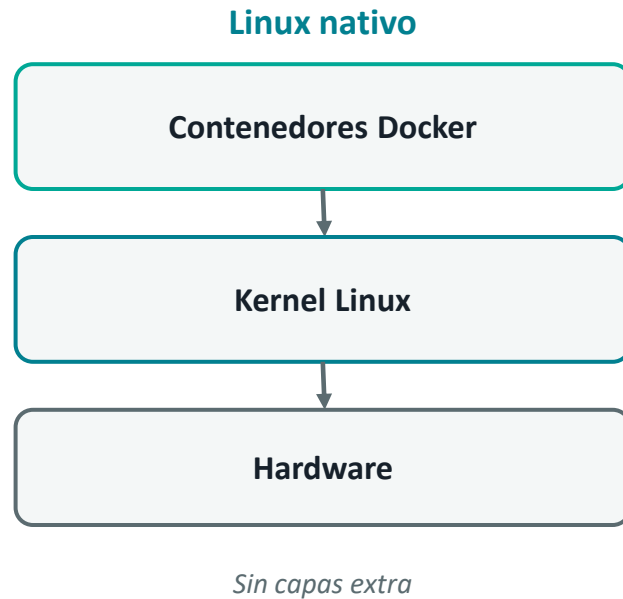
 bash

```
docker run -it --rm -v ${PWD}:/codigo --cap-add=SYS_PTRACE entorno-c
```

Flag	Qué hace
-it	Modo interactivo (-i) + terminal (-t), para escribir dentro del contenedor
--rm	Borra el contenedor automáticamente al salir
-v origen:destino	Monta una carpeta de Windows dentro del contenedor — SIN espacios
--cap-add=SYS_PTRACE	Permite que gdb se enganche a otros procesos dentro del contenedor
entorno-c	El nombre de la imagen que se va a correr

¿Por qué Docker en Windows necesita WSL2?

- Un contenedor Linux necesita un kernel Linux real por debajo — Windows no tiene uno.
- WSL2 corre una máquina virtual ligera con un kernel Linux real adentro.
- Docker Desktop se conecta a ese kernel para poder crear los contenedores.



Tres formas de tener Linux junto a Windows

	Dual boot	VirtualBox	Docker + WSL2
¿Toca hardware real?	Sí, directo	No, simulado	No, vía WSL2
¿Corren juntos?	No, uno a la vez	Sí	Sí
¿Kernel propio completo?	Sí	Sí (duplicado)	Compartido, no duplicado
Peso / velocidad	Rápido, hay que reiniciar	Pesado, lento de arrancar	Ligero, arranca en segundos

Docker es un punto intermedio: no llega a ser tan pesado como una VM completa (no duplica el kernel), pero tampoco toca el hardware directo como el dual boot (sigue necesitando el puente de WSL2 en Windows).

Errores comunes y cómo se resolvieron



Dockerfile.txt en vez de Dockerfile

Docker exige el nombre exacto "Dockerfile", sin extensión.



CMD["zsh"] sin espacio

Cada instrucción del Dockerfile necesita espacio antes del argumento.



"is not valid yet" en apt

Reloj de la VM desincronizado — wsl --shutdown + banderas Check-Valid-Until=false y Check-Date=false.



-v \${PWD} :/codigo (espacio)

El flag -v exige origen:destino pegado, sin espacios.



%cd% dentro de PowerShell

%cd% es de CMD; en PowerShell se usa \${PWD}.



Ejecutar /programa.c o el .c directo

Usa ./binario (no /binario), y compila antes de ejecutar.

Puntos clave para no olvidar

- CMD, PowerShell y WSL son programas distintos — cada uno con su propia sintaxis.
- Un proceso es un programa en ejecución, identificado por su PID.
- `fork()` duplica un proceso: el padre recibe el PID del hijo, el hijo recibe 0.
- Una distro = kernel Linux + herramientas (Debian, Ubuntu, etc.).
- Una imagen Docker es un molde; un contenedor es una instancia corriendo de ese molde.
- Docker en Windows necesita WSL2 porque un contenedor Linux exige un kernel Linux real.
- Contenedores comparten kernel (ligeros); las VMs traen kernel propio (pesadas).
- El flag `-v` necesita origen:destino sin espacios; `%cd%` es de CMD, `${PWD}` es de PowerShell.

Ponte a prueba

Responde sin ver el resto de la presentación:

- 1 ¿Por qué `fork()` no funciona en Windows nativo?
- 2 ¿Cuál es la diferencia entre una imagen y un contenedor de Docker?
- 3 ¿Por qué Docker Desktop en Windows necesita WSL2 para funcionar?
- 4 ¿Qué diferencia hay entre particionar el disco (dual boot) y virtualizar con VirtualBox?
- 5 ¿Qué le indica a un proceso el valor que devuelve `fork()`: 0, positivo o negativo?